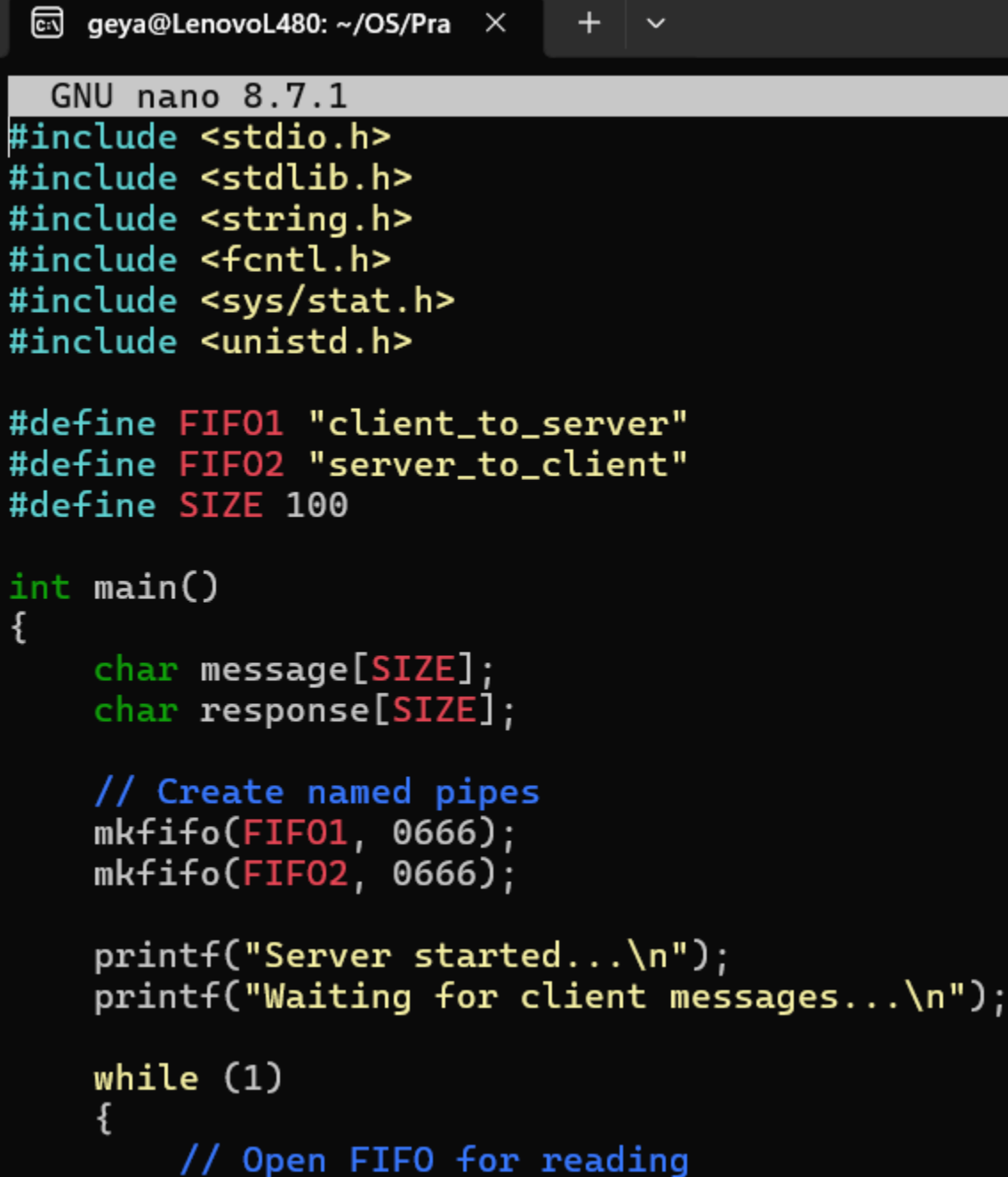


Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Code:



```
geya@LenovoL480: ~/OS/Pra x + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"
#define SIZE 100

int main()
{
    char message[SIZE];
    char response[SIZE];

    // Create named pipes
    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server started...\n");
    printf("Waiting for client messages...\n");

    while (1)
    {
        // Open FIFO for reading
```

```
int fd_read = open(FIFO1, O_RDONLY);

if (fd_read == -1)
{
    perror("Error opening client_to_server");
    exit(1);
}

// Read message from client
int bytes = read(fd_read, message, SIZE - 1);
close(fd_read);

if (bytes > 0)
{
    message[bytes] = '\0';

    printf("Server received: %s\n", message);

    // Exit condition
    if (strcmp(message, "exit") == 0)
    {
        strcpy(response, "Server shutting down...");
        int fd_write = open(FIFO2, O_WRONLY);
        write(fd_write, response, strlen(response) + 1);
        close(fd_write);
    }
}
```

```

        break;
    }

    // Process message
    if (strcmp(message, "hello") == 0)
    {
        strcpy(response, "Hello Client!");
    }
    else if (strcmp(message, "status") == 0)
    {
        strcpy(response, "Server is running normally.");
    }
    else
    {
        snprintf(response, SIZE,
                 "Server processed your message: %.60s", message);
    }

    // Send response to client
    int fd_write = open(FIFO2, O_WRONLY);

    if (fd_write == -1)
    {
        perror("Error opening server_to_client");
        exit(1);

```

```

        write(fd_write, response, strlen(response) + 1);
        close(fd_write);

        printf("Server sent: %s\n", response);
    }
}

// Remove FIFOs
unlink(FIFO1);
unlink(FIFO2);

printf("Server terminated.\n");

return 0;
}

```

Output:

```
priyanka@DESKTOP-HOIS07A:~$ cd ~/OS_Skill/Week-4
priyanka@DESKTOP-HOIS07A:~/OS_Skill/Week-4$ nano src/fifo_server.c
priyanka@DESKTOP-HOIS07A:~/OS_Skill/Week-4$ gcc src/fifo_server.c -o bin/fifo_server
priyanka@DESKTOP-HOIS07A:~/OS_Skill/Week-4$ gcc src/fifo_client.c -o bin/fifo_client
priyanka@DESKTOP-HOIS07A:~/OS_Skill/Week-4$ ./bin/fifo_server
Server waiting for client...
Server received: Hello Server
Server response sent.
priyanka@DESKTOP-HOIS07A:~/OS_Skill/Week-4$ |
```

```
priyanka@DESKTOP-HOIS07A:~$ cd ~/OS_Skill/Week-4
priyanka@DESKTOP-HOIS07A:~/OS_Skill/Week-4$ ./bin/fifo_client
Server response: Message processed by server
priyanka@DESKTOP-HOIS07A:~/OS_Skill/Week-4$
```